Notion d’Algorithmique

Algorithme

- Algorithme vient du nom du mathématicien « **Al-Khawarizmi** » latinisé en ***Algoritmi***. *Née en 780 et mort* vers 850 à Bagdad
- Al-Khawarizmi a vécu au 9ème siècle et était membre de l'académie des sciences à Bagdad.

- **Définition :** Un al**gor**ithme est :
  - une suite finie d'instructions que l'on applique à un nombre fini de données dans un ordre précis pour arriver à un résultat.
  - une suite ordonnée d'instructions qui indique la démarche à suivre pour résoudre une série de problèmes équivalents.

Algorithme

Rôle fondamental de l'Algorithme:

- Il prend des données en entrée.
- Il exprime un traitement particulier.
- Il fournit des données en sortie.

Comment trouver un algorithme ?

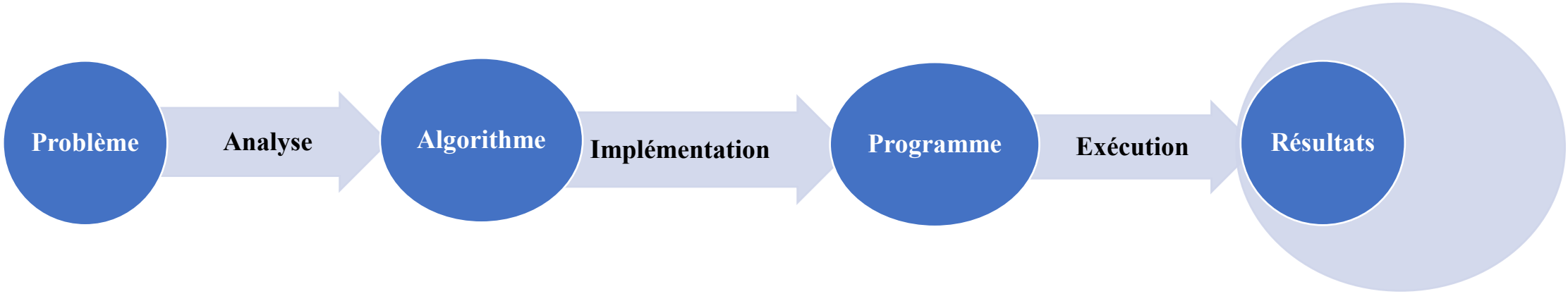
Questions préliminaires à se poser avant la conception d'un algorithme :

- Quel est le résultat recherché ?
- Quelles sont les données disponibles ou les informations d'entrée ?
- Quelles sont instructions à appliquer à ces données pour aboutir au résultat attendu ?

Algorithme

- Un algorithme ne peut pas être exécuté directement par une machine :
  - il est écrit dans un langage abstrait et conceptuel,
  - il n'est pas exprimé sous la forme de codes binaires que le processeur peut interpréter.
- Un algorithme doit être traduit dans un langage de programmation
- **Définition : Un programme** est :
  - la traduction d'un algorithme dans un langage informatique particulier et parfaitement précis.
  - une série d'instructions qui réalise (implémente) un algorithme
  - Un programme n'est pas directement interprété par la machine, il doit être traduit en langage machine. Cette traduction est faite automatiquement par un programme appelé « **COMPILATEUR** » ou « **INTERPRETEUR** »

Processus de résolution de problèmes



Algorithme

- L’écriture algorithmique est un travail de programmation à visée universelle.
- Un algorithme ne dépend pas du langage dans lequel il est implanté.
- Un algorithme ne dépend pas de la machine qui exécutera le programme correspondant.
- L'algorithmique permet d’expliquer comment faire un travail sans la moindre ambiguïté, en utilisant un langage simple composé d'instructions élémentaires.

Propriétés et Qualités d’un Algorithme

Un algorithme doit obligatoirement :

- Avoir un nombre fini d’étapes.
- Avoir un nombre fini d’opérations par étape.
- Se terminer après un nombre fini d’opérations.
- Fournir un résultat.
- Chaque opération doit être définie rigoureusement et sans ambiguïté.
- Chaque opération doit être effective (réalisable par une machine).

Algorithmique

L’algorithmique s’intéresse à l'art de construire des algorithmes et à caractériser leurs qualités :

- **Validité :** Aptitude à réaliser exactement la tâche pour laquelle il a été conçu.
- **Robustesse :** Aptitude à se protéger des conditions anormales d’utilisation.
- **Efficacité :** Aptitude à utiliser de manière optimale les ressources du matériel qui l’exécute.
- **Réutilisabilité :** Aptitude à être réutilisé pour résoudre des tâches équivalentes.
- **Complexité** : Le nombre d’instructions élémentaires à exécuter pour réaliser la tâche. Cette notion s'intéresse à l'espace et au temps nécessaire pour que l'algorithme atteigne le résultat escompté.

Algorithme Equation_Premier_Degre

Début

```
Ecrire(  " Entrer la valeur de a : ")
Lire (a)
Ecrire(  " Entrer la valeur de b : ")
Lire (b)
Si a <>0 alors
        Ecrire(  " la solution est :"  , -b/a)
sinon
    si b=0 alors
        Ecrire ("l'équation est insoluble")
    sinon
        Ecrire (" tout réel est solution")
    Finsi
Finsi
```

Fin

```
print("Entrez la valeur de a")
a=int(input())
print("Entrez la valeur de b")
b=int(input())
if a!=0:
    print("la valeur de x est " , -b/a)
else:
    if b==0:
        print("Tout réel est solution ")
    else:
        print("Pas de soulution")
```

Structure générale d'un algorithme

La représentation d'un algorithme se fait généralement en pseudo-code. Un algorithme comporte trois parties principales :

- **L'en-tête :** Sert à donner un nom à l'algorithme, précédé du mot Algorithme.
- **La partie déclarative :** On y déclare les différents objets que l'algorithme utilise (constantes, variables).
- **Le corps de l'algorithme :** Contient les instructions. Il est délimité par les mots Début et Fin.



Algorithme Nom_algorithme — **L'entête de l'algorithme**

Constantes: déclaration des constantes
Variables: Déclaration des variables — **La partie declaration**

Début
Bloc d'instructions
Fin — **La partie traitement**

Structure générale d'un algorithme

Algorithme Nom_algorithme — **L'entête de l'algorithme**

Constantes: déclaration des constantes
Variables: Déclaration des variables — **La partie declaration**

Début
Bloc d'instructions
Fin — **La partie traitement**

```
Algorithme surface_cercle

Constantes: pi ← 3,14
Variables: rayon,surface : Réel

Début
Ecrire( " Veuillez saisir le rayon ")
Lire ( rayon)
surface ← rayon* rayon*pi
Ecrire(surface)
Fin
```

Variables

- Une variable est un **emplacement mémoire** (boîte étiquetée).
- Son contenu peut **changer** au cours d'un programme.
- **Règle fondamentale :** La variable doit être déclarée avant d'être utilisée.

Déclaration des Variables :

- Elle doit comporter le nom (identificateur) et le type.
- **Syntaxe :** Variable **identificateur** : type.
- **Identificateur :** Nom donné à une variable . Il doit obligatoirement commencer par une lettre, suivie d'une suite de lettres et de chiffres, sans contenir d'espace.
- **Type :** L'ensemble des valeurs qu'une variable peut prendre (ex: le type logique/booléen peut prendre Vrai ou Faux).

Variables

- Une variable est un **emplacement mémoire** (boîte étiquetée).
- Son contenu peut **changer** au cours d'un programme.
- **Règle fondamentale :** La variable doit être déclarée avant d'être utilisée.

Déclaration des Variables :

- Elle doit comporter le nom (identificateur) et le type:

Syntaxe : Variable **identificateur** : type

- **Identificateur :** Nom donné à une variable . Il doit obligatoirement commencer par une lettre, suivie d'une suite de lettres et de chiffres, sans contenir d'espace.
- **Type :** L'ensemble des valeurs qu'une variable peut prendre

Types de Variables

Cinq types de base:

- Le type entier (Positif et négatif)
- Le type réel
- Le type booléen (vrai, faux)
- Le type chaine de caractères
- Le type caractère (une lettre, un chiffre ou un symbole)

Types de Variables

Type	Description
Entier	Désigne les nombres entiers positifs et négatifs.
Réel	Désigne les valeurs numériques réelles.
Logique / Booléen (BOOLEEN)	Vrai ou Faux (True ou False, 1 ou 0).
Caractère (CARACTERE)	Un seul caractère.
Chaîne de Caractères (CHAINE)	Séquence de caractères.

Les Constantes

- Une constante correspond aussi à un emplacement mémoire.
- Sa valeur stockée ne sera jamais modifiée au cours du programme.
- Syntaxe :

Constante NOM_DE_LA_CONSTANTE = valeur

- Exemple : Constante PI = 3.14.

Affectation

- L'affectation est l'opération la plus utilisée dans un algorithme, elle permet d'attribuer une valeur à une variable ou une constante
- Elle est symbolisée en algorithmique par ←.

Syntaxe : Variable ← valeur

- À gauche de la flèche, un nom de variable, et uniquement cela
- À la droite de la flèche on trouve soit une valeur ou une expression
- **Exemples:**
  - pi ← 3,14
  - message ← " Bonjour "
  - rayon ← 5
  - reponse ← Vrai
  - Choix ← " N "

NB : En informatique, une variable possède à un moment donné une valeur et une seule.

Exercice 1 :

- Trouver les valeurs des variables A, B et C après exécution des instructions suivantes pour les deux algorithms suivants :

Variables: A, B, C **: Entier**
Début
A ← 6
B ← 4
C ← A + B
A ←3
C ← B – A
Fin

Variables: A, B :**Entier**
Début
A ← 5
B ← A + 4
A ← A + 1
B ← A – 4
Fin

Exercice 2 :

- Trouver les valeurs des variables A et B après exécution des instructions suivantes :

Variables A, B :**Entier**

Début

A ← 5

B ← 2

A ← B

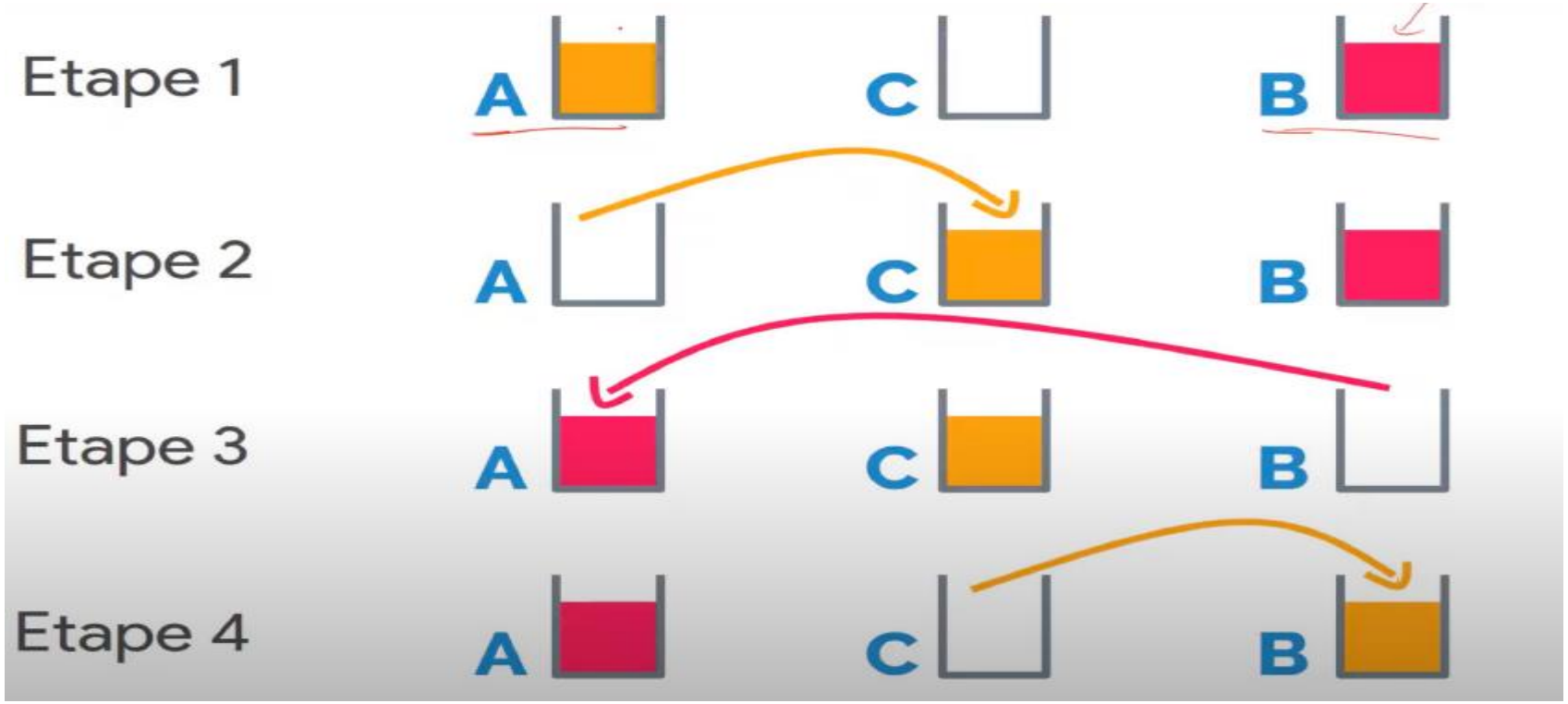
B ← A

Fin

- Est-ce que les deux dernières instructions permettent-elles d'échanger les deux valeurs de B et A?

Exercice 3:

- Ecrire un algorithme permettant d'échanger les valeurs des variables A, B, ce quel que soit leur contenu préalable:



Solution Exercice 3

Variables A, B :**Entier**

Début

C ← A

A ← B

B ← C

Fin

Les opérateurs

Opérateurs arithmétiques

Symbole	Opération
+	Addition
-	Soustraction
*	Multiplication
/	Division
Div	Quotient de la division entière.
Mod	Modulo (reste de la division euclidienne).
^	Puissance.

Exemple:
Variables: A, B **en Entier**
Début
A ← 7
B ← 3
C ← A div B
D ← A mod B
Fin

- Utiliser les parenthèses, avec les mêmes règles qu'en mathématiques.
- La multiplication et la division ont la priorité sur l'addition et la soustraction.
- Les parenthèses ne sont ainsi utiles que pour modifier cette priorité naturelle.

Opérateurs **de comparaison** –Relationnels-

Symbole	Signification
=	Égal
< >	Différent
<	Inférieur
<=	Inférieur ou égal
>	Supérieur
>=	Supérieur ou égal

Le résultat d'une opération de comparaison est de type **booleen**

Opérateurs **de concaténation** -sur les Chaînes-

- L'opérateur **&** permet de **concaténer** deux chaînes de caractères

Algorithme concatener
Variables: nom,prenom,nom_prenom : chaine
Début
nom ← **" Alice "**
prenom ← **" Bob "**
nom_prenom ← **nom&prenom**
Fin

Opérateurs Logiques

ET , OU , NON , OU exclusif (XOR)

X	Y	X **OU** Y
0	0	0
0	1	1
1	0	1
1	1	1

X	Y	X **ET** Y
0	0	0
0	1	0
1	0	0
1	1	1

X	Y	X **XOR** Y
0	0	0
0	1	1
1	0	1
1	1	0

X	**NON** X
0	1
1	0

Les fonctions d’entrées – sorties “Lecture-écriture”

- Pour assurer la communication entre l’homme et la machine, les algorithmes ont besoin de données à traiter qui seront communiquées au programme par l’intermédiaire du clavier, Les résultats trouvés doivent être affiché à l’écran. à Ces deux opérations sont assurées par les 2 fonctions: **Lire** et **Ecrire**

Instruction Lire

La fonction Lire permet au programme de lire les données à partir du clavier.

- **Syntaxe:**

Lire (variable1,variable2,….)

➢Dès que le programme rencontre une instruction lire, l'exécution s'arrête en attendant la saisie d'une valeur au clavier

Instruction Ecrire

- Elle permet d’afficher des informations à l'écran.
- Il est conseillé d’écrire des libellés à l’écran avant de lire une variable, pour prévenir l’utilisateur de ce qu’il doit saisir.

Syntaxe : Ecrire (expression)

- **expression** : peut être une valeur, un résultat, un message, ou le contenu d'une variable

Exercice

- Ecrire un algorithme qui demande un nombre à l'utilisateur, puis qui calcule et affiche le carré de ce nombre.

Solution

Algorithme calcul_carre

Variables: nb,carre : Réel

Début

Ecrire(" Veuillez saisir un nombre ")

Lire (nb)

carre ← nb* nb

Ecrire(**" le carré de ",** nb, **" est : ",** carre **)**

Fin

Les structures conditionnelles: Si

- La structure conditionnelle Si permet d'exécuter certaines instructions **uniquement si une condition donnée est vraie**. Elle peut éventuellement inclure une partie Sinon pour exécuter d'autres instructions lorsque la condition est fausse.
- Structure de base :

```
Si condition Alors
    instructions_si_vrai
Sinon
    instructions_si_faux
FinSi
```

- **condition** : expression qui peut être vraie ou fausse.
- **instructions_si_vrai** : exécutées si la condition est vraie.
- **instructions_si_faux** : exécutées si la condition est fausse.

Exemple 1

- **Ecrire un algorithme qui affiche la valeur absolue d'un réel x**

Solutions

Algorithme valeur_absolue
Variables: x,y : Reel
Début
Ecrire(" Veuillez saisir un nombre")
Lire (x)
y ← x
Si (x<0) **alors**
y ← - x
Finsi
Ecrire(" La valeur absolue de " , **x** , " est " , **y**)
Fin

Algorithme valeur_absolue
Variables: x,y : Reel
Début
Ecrire(" Veuillez saisir un nombre")
Lire (x)
Si (x > 0) **alors**
y ← x
Sinon
y ← - x
Finsi
Ecrire(" La valeur absolue de " , **x** , " est" , **y**)
Fin

Exemple 2

- **Ecrire un algorithme qui affiche le max de deux réel x et y.**

Solution

- **Algorithme** max_valeur
- **Variables**: x , y, max : Reel
- **Début**
- **Ecrire**(" Veuillez saisir 2 nombres**")**
- Lire (x , y)
- **Si** (x >= y) **alors**
- max ← x
- **Sinon**
- max ← y
- **Finsi**
- **Ecrire**(" le max est **" , max)**
- **Fin**

Autre forme de SI

Si (Condition **1) alors**

Si (Condition **2) alors**

Bloc d'instructions **A**

Sinon

Bloc d'instructions **B**

Finsi

Sinon

Si (Condition **3) alors**

Bloc d'instructions **C**

Finsi

Finsi

Exemple

- Ecrire un algorithme qui demande à l'utilisateur de saisir un chiffre et affiche ensuite, si le nombre est positif, négatif ou nul

Solution

```
Algorithme signe
Variables: x: Réel
Début
Ecrire( " Veuillez un nombre")
Lire ( x )
Si (x > 0 ) alors
        Ecrire( " Positif")
Sinon
        Si (x = 0 ) alors
                Ecrire( " Nul")
        Sinon
                Ecrire( " Négatif")
        Finsi
Finsi
Fin
```

Exercice

- Ecrire un algorithme qui affiche le max de 3 réel x , y et z

Solution

```
Algorithme max_valeur
Variables: x , y, z , max : Reel
Début
Ecrire( " Veuillez saisir 3 nombres")
Lire ( x , y , z )
Si (x >= y) alors
        Si (x >= z) alors
                max ← x
        Sinon
                max ← z
        Finsi
Sinon
        Si (y >= z) alors
                max ← y
        Sinon
                max ← z
        Finsi
Finsi
Ecrire( " le max est " , max)
Fin
```

Structure conditionnelle Selon

- comparer une seule variable à une liste de valeurs connues à l'avance
- La variable de la structure de selon ne peut pas être de type Réel
- **Syntaxe :**

Selon Variable

Valeur 1 : **Instruction1**

Valeur 2 : **Instruction2**

……

……

……

Valeur n : **Instruction n**

Autrement : **autre instruction**

Finselon

Exemple

- Ecrire un algorithme qui demande à l'utilisateur de saisir un entier compris entre 1 et 7 et affiche ensuite le jour correspondant

Solution:

Algorithme Jour_semaine
Variables: x: entier
Début
Ecrire(" Veuillez saisir un nombre compris entre 1 et 7")
Lire (x)
Selon x
1 : **Ecrire**(" Lundi")
2 : **Ecrire**(" Mardi")
3 : **Ecrire**(" Mercredi")
4 : **Ecrire**(" Jeudi")
5 : **Ecrire**(" Vendredi")
6 : **Ecrire**(" Samedi")
7 : **Ecrire**(" Dimanche")
Autrement : **Ecrire**(" Le nombre doit etre compris entre 1 et 7")
Finselon
Fin

Organigramme

- Un organigramme est une représentation graphique de l'algorithme.
- La norme adoptée dans ce cours est : **ISO5807**

Symboles de l’organigramme



Début ou fin



Entrée - Sortie



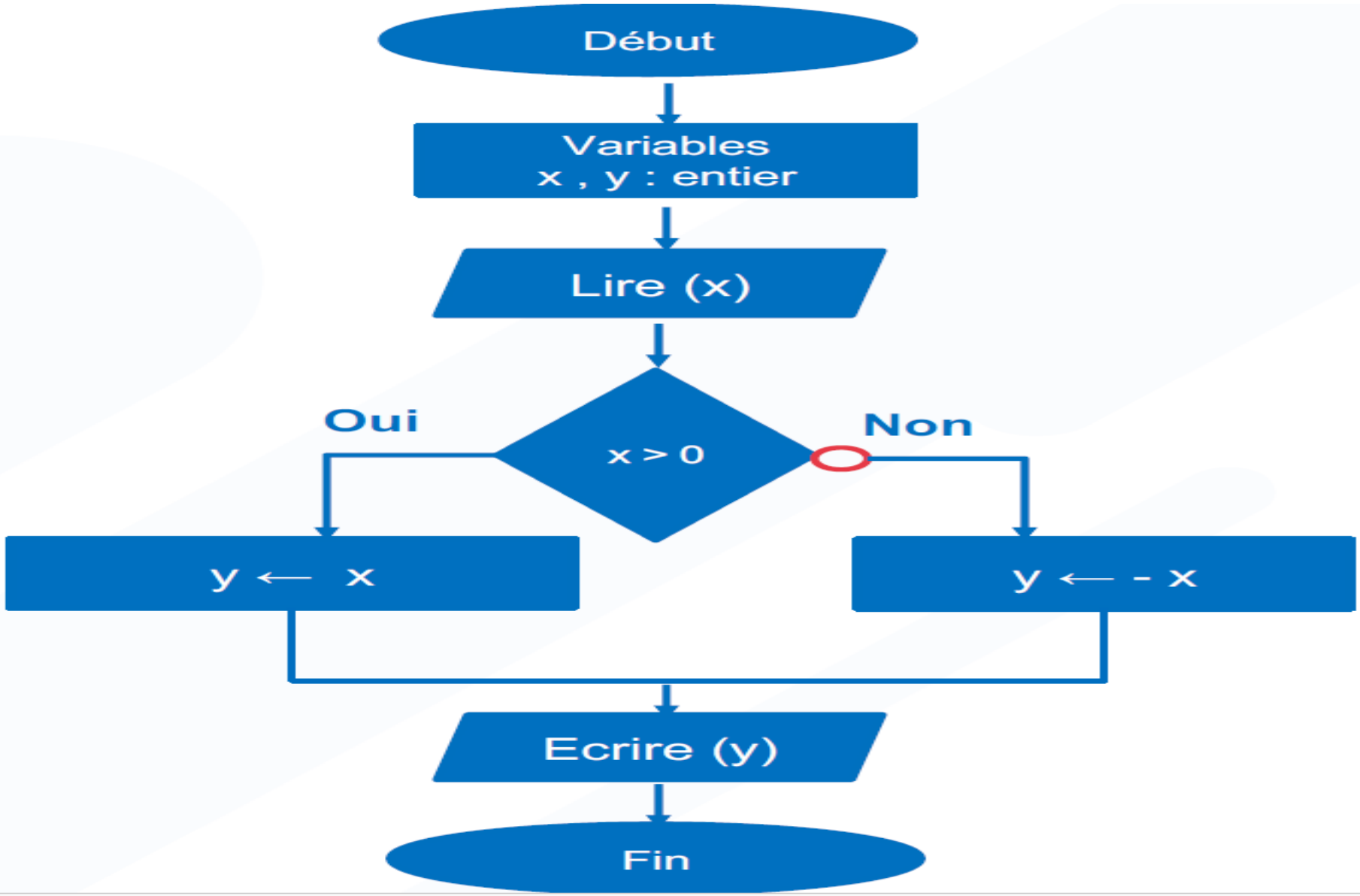
Opération ou groupement d’opérations





Liaison

Organigramme de la valeur absolu



Les Structures Répétitives : Les boucles

Les Structures Répétitives : Les boucles

- Une structure répétitive (Boucle) est un ensemble d'actions qui se répète toujours dans une ordre précis, un nombre **finis** de fois
- En algorithmique nous avons trois structures:
  - La boucle **Tantque…. Faire**
  - La boucle **Repeter ….Jusqu'à**
  - La boucle **Pour……..Faire**

La boucle Tantque…. Faire

Syntaxe:

TantQue (condition) **Faire**

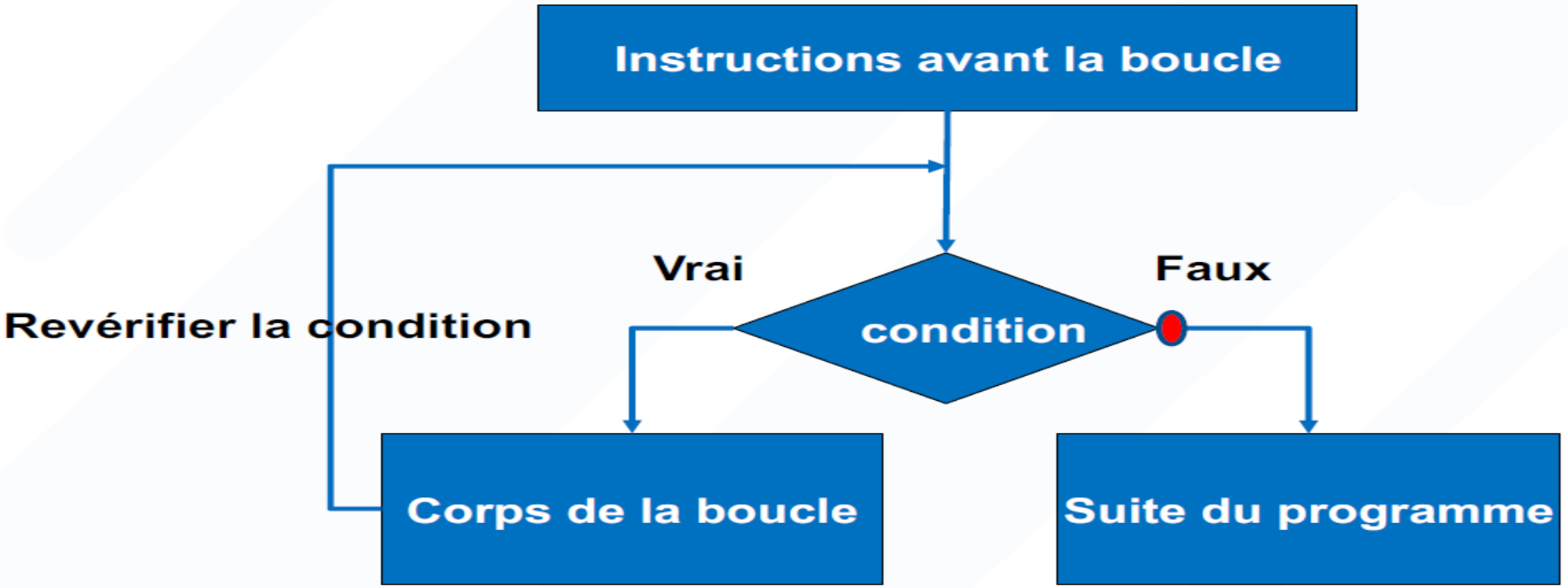
Instructions

FinTantQue

La boucle Tantque…. Faire

- La condition (condition de contrôle de la boucle) est évaluée avant chaque itération.
  - Si la condition est vraie, le programme exécute le corps de la boucle, puis revient vérifier à nouveau la condition. Tant que celle-ci reste vraie, le corps de la boucle continue d'être exécuté.
  - Si la condition devient fausse, l'exécution de la boucle s'arrête et le programme poursuit avec les instructions situées après le mot-clé Fintantque.
- Le nombre d'itérations d'une boucle TantQue n'est pas connu à l'avance. Il dépend de l'évolution de la condition de contrôle.
- Il est essentiel qu'une instruction du corps de la boucle modifie cette condition pour qu'elle passe de vraie à fausse après un certain nombre d'itérations ; sinon, la boucle s'exécutera infiniment.

Organigramme de la boucle Tantque



Exemple1 : Afficher tout les nombres pairs inferieur ou égale à 100

```
Algorithme paire
Variables : i: entier
Debut
i ←0
Tantque (i <= 100) Faire
Ecrire(i)
i ←i+2
Fintantque
Ecrire( " end" )
Fin
```

Exemple 2 : Calculer la somme des N premiers nombres entiersS=1+2+3+4……..+N

```
Algorithme Somme
Variables : S,i,n: entier
Debut
Ecrire (" Entrez la valeur de N " )
Lire (N)
S ← 0
i ← 1
Tantque (i <= N) Faire
    S ← S + i
    i ← i+1
Fintantque
Ecrire (" La somme de ", N, " Premiers entier est " , S)
```

Exemple 3 : Contrôle de saisie d'une lettre majuscule jusqu'à ce que le caractère entré soit valable

```
Algorithme  Majuscule
Variable c : caractere
Debut
Ecrire (" Entrez une lettre majuscule ")
Lire (c)
TantQue (c < 'A' ou c > 'Z') Faire
    Ecrire ("Saisie erronee. Recommencez")
    Lire (c)
FinTantQue
Ecrire ("Saisie valable")
Fin
```

La boucle Repeter…. Jusqu'à

Syntaxe:

Repeter

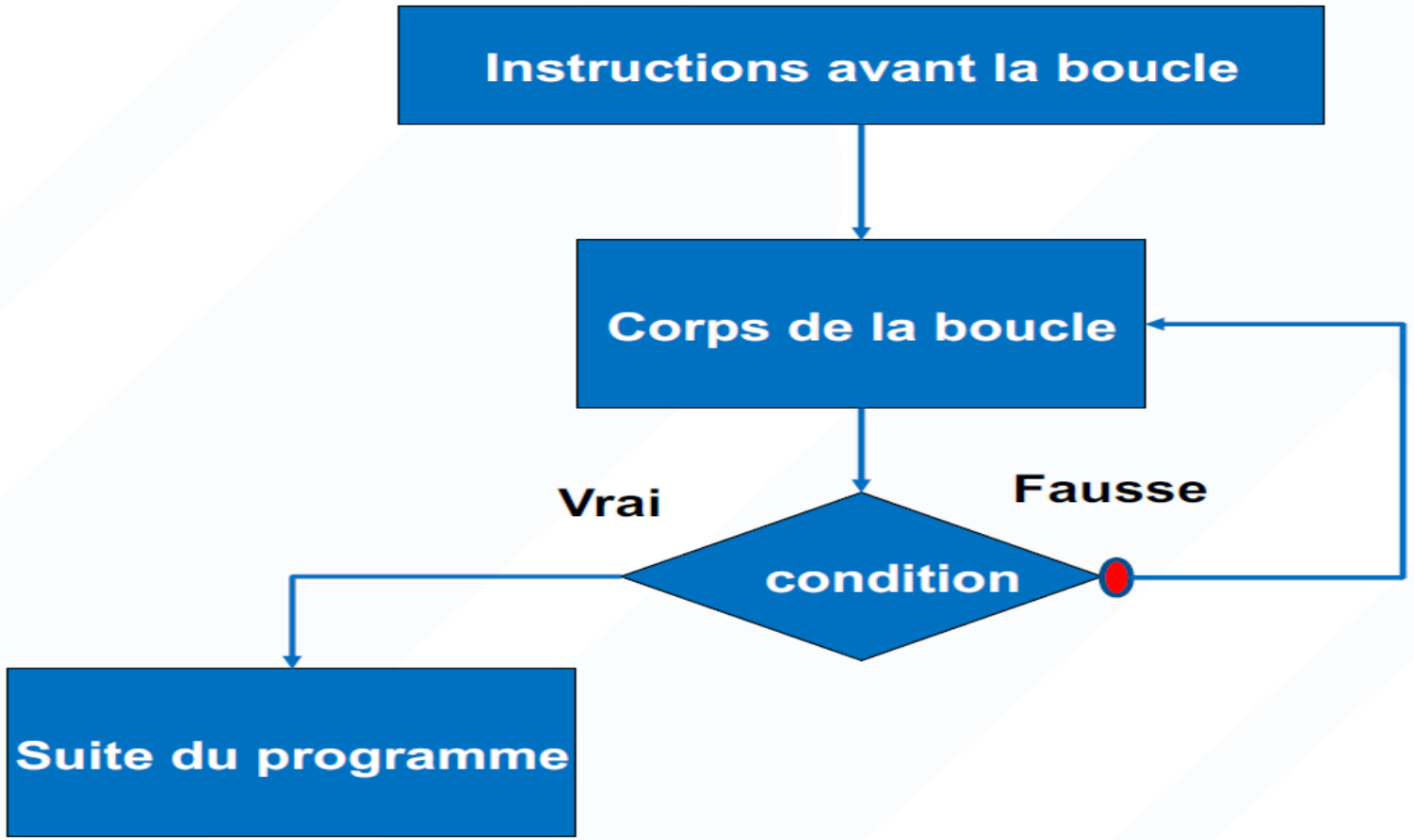
Instructions

Jusqu'à (condition)

La boucle Repeter…. Jusqu'à

- La boucle Répéter est utilisée lorsque l'on souhaite exécuter plusieurs fois une série d'instructions jusqu'à ce qu'une condition donnée devienne vraie.
- Dans une boucle Répéter, le test de la condition est effectué à la fin de chaque itération. Cela signifie que le corps de la boucle est exécuté au moins une fois, même si la condition est déjà vraie au départ.
- À l'inverse, dans une boucle TantQue, la condition est testée avant l'exécution du corps. Par conséquent, il est possible que le corps de la boucle ne soit jamais exécuté si la condition est fausse dès le début.
- La condition de la boucle Tantque est la négation de la condition de la boucle Repeter

Organigramme de la boucle Repeter



Exemple1 : Afficher tout les nombres pairs inferieur ou égale à 100

```
Algorithme paire
Variable  i: entier
Debut
i ← 0
Repeter
Ecrire(i)
      i ← i+2
      Jusquà (i > 100)
Ecrire( " c'est fini " )
Fin
```

La condition de la boucle **Tantque** est la négation de la condition de la boucle **Repeter**

Exemple 2 : Calculer la somme des N premiers nombres entiersS=1+2+3+4……..+N

```
Algorithme Somme
Variables : S,i,n: entier
Debut
Ecrire (" Entrez la valeur de N " )
Lire (N)
S ← 0
i  ← 0
Repeter
      S  ← S + i
      i ← i+1
Jusqu'à (i > N)
Ecrire (" La somme de ", N, " Premiers entier est " , S)
```

Exemple 3 : Contrôle de saisie d'une lettre majuscule jusqu'à ce que le caractère entré soit valable

```
Algorithme  Majuscule
Variable c : caractere
Debut
Ecrire (" Entrez une lettre majuscule ")
Lire (c)
Repeter
      Ecrire (" Entrez une lettre majuscule ")
      Lire (c)
jusqu'a ( c>= 'A' et c<= ' Z' )
Fin
```

La boucle Pour….. Faire

Syntaxe:

Pour i ← allant de val_initial **à** val_finale **Faire**

    Instructions

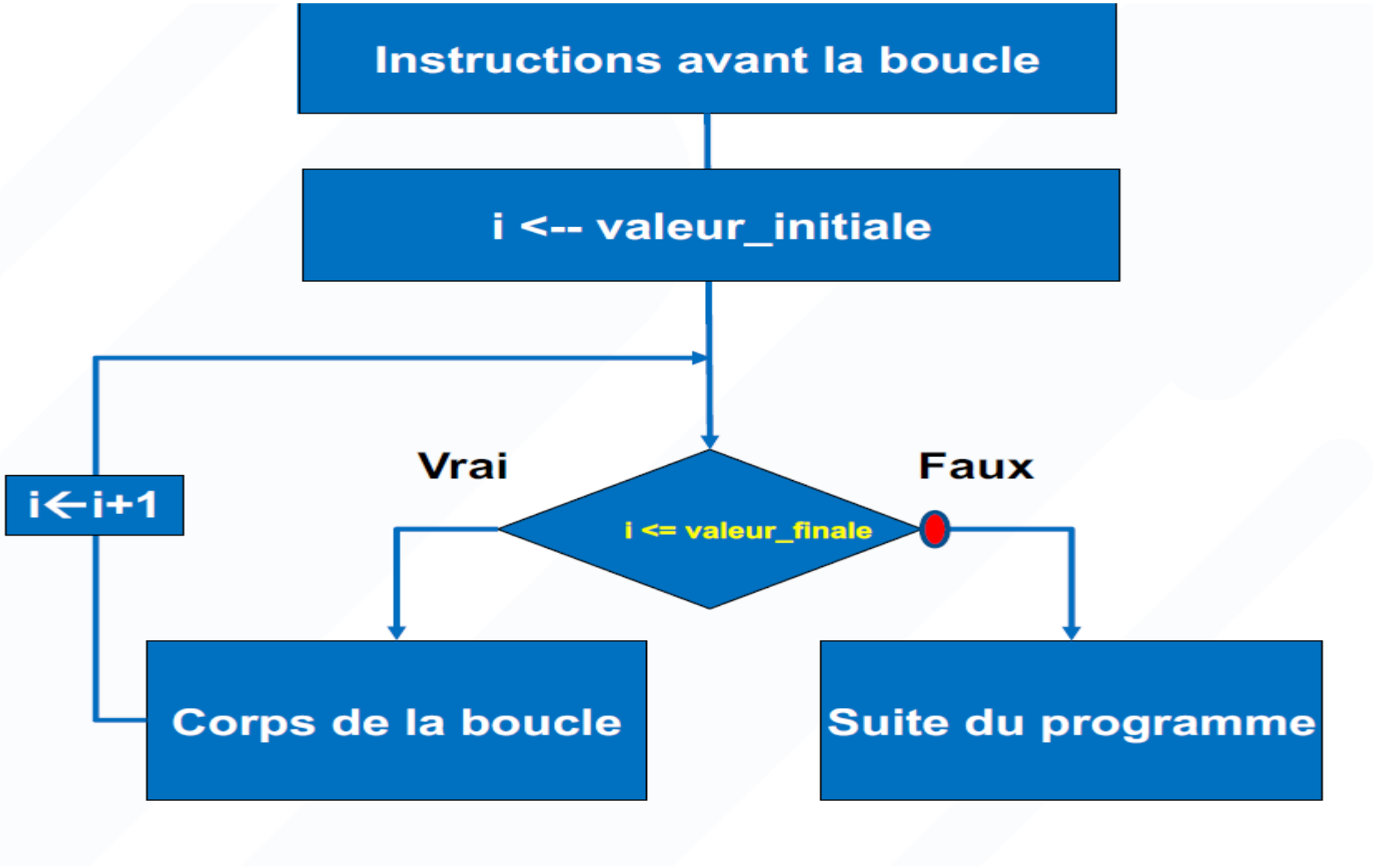
FinPour

La boucle Pour….. Faire

- La structure « **pour** » est utilisée lorsqu'un bloc d'instructions doit être exécuté plusieurs fois et que le nombre d'itérations est connu à l'avance.
- La variable **i**, appelée compteur de boucle, est une variable entière qui doit être déclarée avant son utilisation.

- Remarques:
  - La boucle **Pour** est simplement une version de la boucle **Tant Que**, utilisée lorsque le nombre précis d'itérations est connu dès le départ.
  - Par conséquent, ce qu'on peut exprimé avec une boucle **Pour** peut être réécrit en utilisant une boucle **Tant Que**, aLors que l'inverse n'est pas possible.

Organigramme de la boucle Pour….. Faire



Exemple1 : Afficher tout les nombres pairs inferieur ou égale à 100

```
Algorithme paire
Variable  i: entier
Debut
Pour i ←  allant de 0 à 100 pas 2 Faire
Ecrire(i)
FinPour
Fin
```

Exemple1 : Afficher tout les nombres inferieur ou égale à 100

```
Algorithme paire
Variable  i: entier
Debut
Pour i ←  allant de 0 à 100 Faire
Ecrire(i)
FinPour
Fin
```

Exemple 2 : Calculer la somme des N premiers nombres entiersS=1+2+3+4……..+N

```
Algorithme Somme
Variables : S,i,N: entier
Debut
Ecrire (" Entrez la valeur de N " )
Lire (N)
S ← 0
Pour i ← allant de 1 à N Faire
    S ← S + i
FinPour
Ecrire (" La somme de ", N, " Premiers entier est " , S)
```

Exercice 1 : Calculer x^m ou x est un reel non nul et m un entier positif ou nul

```
Algorithme Puissance
Variables x, puiss : reel
Variables m, i : entier
Debut
Ecrire (" Entrez la valeur de x ")
Lire (x)
Ecrire (" Entrez la valeur de m ")
Lire (m)
puiss ← 1
Pour i ←  allant de 1 à m
      puiss← puiss*x
FinPour
Ecrire (x, " a la puissance ", m, " est egal a ", puiss)
Fin
```

Exemple 2 : Calculer la somme des N premiers nombres entiers S=1+2+3+4……+N

```
Algorithme Somme
Variables : S,i,n: entier
Debut
Ecrire (" Entrez la valeur de N " )
Lire (N)
S ← 0
Pour i ← allant de 1 à N Faire
    S ← S + i
FinPour
Ecrire (" La somme de ", N, " Premiers entier est " , S)
```

Les fonctions

Problématiques

- **Probléme : Répétition du Code**
- **Solution : Ecrire le code une fois et l'appeler plusieurs fois**
  - **Les fonctions**

Introduction

- Les fonctions et les procédures sont des sous programmes (un ensemble d'instructions) auxquels on peut faire référence à l'intérieur d'un programme.
  - Eviter les répétitions
  - Structuration et une meilleure lisibilité des programmes
  - Faciliter la maintenance du code (il suffit de modifier une seule fois)
  - Réutilisées les procédures et fonctions dans d'autres programmes

Introduction et Définition

Définition :

Une fonction est un sous-programme qui reçoit des valeurs en entrée lors de son appel et retourne un résultat à la fin de son exécution.

- **Déclaration :**
  Une fonction est **déclarée** à l'aide du mot-clé « **Fonction** », en précisant :
  - son **nom** (ou **identificateur**), qui doit respecter les mêmes règles que celles des noms de variables,
  - la **liste des paramètres** (ou **arguments**) accompagnés de leur **type**,
  - le **type du résultat** qu'elle retourne,
  - l'instruction **retourne**, utilisée pour **renvoyer la valeur du résultat**.

Déclaration d'une fonction

Fonction nom_fonction(par1 : type, par2 : type, ...) : **type_de_la_valeur_retournée**

Variables

Début

    Instructions constituant le corps de la fonction

    retourne résultat

FinFonction

Calcul de la somme de deux nombres

```
Fonction somme(a : entier, b : entier) : entier
Variables
    resultat : entier
Début
    resultat ← a + b
    retourne resultat
FinFonction
```

Calcul de a^n

```
Fonction puissance (a:entier, n:entier) : entier
Variables  P,i:entier
Début
      P  ← 1
      Pour i allant de 1 à n faire
            P  ← P*a
      FinPour
retourne P
FinFonction
```

Appel d’une fonction

Calcul de la somme de deux nombres

```
Algorithme CalculSomme

Fonction somme(a : entier, b : entier) : entier
Variables
    resultat : entier
Début
    resultat ← a + b
    retourne resultat
FinFonction
```

```
Variables
    x, y, z : entier
Début
    Ecrire ( "Entrez le premier nombre :)
    Lire ( x)
    Ecrire ( "Entrez le deuxième nombre :“)
    Lire ( y)

    z ← somme(x, y)  // Appel de la fonction somme

    Ecrire ( "La somme est : ", z)
FinAlgorithme
```

Appel d'une fonction

Calcul de $x^n + y^m$

```
Algorithme somme_puissance
Variables : x,y,n,m,R :entier
Fonction puissance (a:entier, b:entier) : entier
Variables: P,i:entier
Début
P ← 1
    Pour i allant de 1 à b faire
        P ← P*a
    FinPour
retourne P
FinFonction
Début
    Ecrire ("Entrez la valeur de
    x,y,n et m ")
    Lire(x,y,n,m)
    R ← puissance (x,n)+
    puissance (y,m)
    Ecrire (" le résultat est " , R)
Fin
```

Variables globales

Variables locales

Exercice 1

Ecrire une fonction qui calcule la somme des nombres paires inférieurs à un entier n saisi par l'utilisateur. Et un programme principal qui appelle la fonction avec la valeur de n=1000 puis la valeur n=2000

Exercice 1: proposition de solution

Algorithme Somme_paires
Fonction Somme_paire (N:entier) : entier
Variables: S,i :entier
Début
S ← 0
i ← 0
Tantque (i <= N) Faire
S ← S + i
i ← i+2
Fintantque
retourne S
Fin

Début

Ecrire (" La somme des nombres paires inférieurs à 1000 est " , **Somme_paire(1000)**)

Ecrire (" La somme des nombres paires inférieurs à 2000 est " , **Somme_paire(2000)**)
Fin

Exercice 2

- Ecrire une fonction paire qui retourne Vrai si le nombre passé en paramètre est pair et Faux dans le cas contraire.
- Appeler la fonction plusieurs fois dans le programme et demander à chaque fois à l'utilisateur s'il veux refaire la saisie

Correction

```
Algorithme pair_impair
Variables : x: entier
            choix:caractère
Fonction paire (a:entier) : Booleen
            Variables: R: booleen
Début
    si (a mod 2=0) alors
            R ← Vrai
    Sinon
            R ← Faux
    Finsi
    retourner R
Fin
```

```
Début
Repeter
    Ecrire ("Entrez un entier")
    Lire (x)
    Si (paire(x)=Vrai) alors
        Ecrire (x, " est pair ")
    Sinon
        Ecrire (x, " est impair ")
    Finsi
    Ecrire (" pour continuez tapez O pour sortir tapez une
    autre touche")
    Lire (choix)
    Jusqu'à(choix <>'O' et choix <>'o' )
Fin
```

Les procedures

Procédure

- Une procédure est un sous-programme qui effectue des instructions mais ne retourne pas de valeur unique.
  - Elle peut modifier des variables ou afficher des résultats, mais ne renvoie pas directement une valeur comme une fonction.
  - Si plusieurs résultats doivent être transmis, ils sont généralement passés via des paramètres.

Déclaration d'une procédure

Procedure nom_procedure (par1 : type, par2 : type, ...)

Variables

Début

Instructions

Fin

Exemple : Table de multiplication

Procedure table_multiplication (n:entier)
Variable i:entier
Début
Pour i allant de 1 à **10** faire
Ecrire (n,"*", i , "=", n*i)
FinPour
Fin

Appel d une procédure

- L'appel de la procédure se fait dans le programme principal
- Contrairement à l'appel d'une fonction, on ne peut pas affecter la procédure à une variable ou l'afficher.

Exemple : Table de multiplication

```
Algorithme afficher_table
Variable   x: entier
Procedure table_multiplication (n:entier)
Variable  i:entier
Début
Pour i allant de 1 à 10 faire
     Ecrire (n,"*", i , "=", n*i)
FinPour
Fin
```

```
Début
Ecrire (" Entrer un réel ")
Lire (x)
table_multiplication(x)
Fin
```

Paramètres d'une procédure

- Les paramètres servent à échanger des données entre le programme principale et la procédure appelée.
- Les paramètres placés dans la déclaration d'une procédure sont appelés **paramètres formels**. Ces paramètres peuvent prendre toutes les valeurs possibles mais ils sont abstraits (n'existent pas réellement)
- Les paramètres placés dans l'appel d'une procédure sont appelés **paramètres effectifs ou réels**. ils contiennent les valeurs pour effectuer le traitement
- Le nombre de paramètres effectifs doit être égal au nombre de paramètres
- formels. L'ordre et le type des paramètres doivent être identique.

Types de paramètres

Paramètres d'entrée (IN):

- ils correspondent aux valeurs que vous souhaitez transmettre à la procédure.
- Toute modification des paramètres à l'intérieur de la procédure n'aura aucune influence sur les variables d'origine.
- **la valeur de ce parametre à l'extérieur de la procédure ne change pas**.
- **Exemple :**

```
Procédure afficherMessage(message : chaine)
Début
    Ecrire message
FinProcédure
```

- Ici, message est un **paramètre d'entrée**.
- Les paramètres d'Entrée/sortie sont précédés par « E »

Types de parametres

Paramètres de sortie (OUT)

- La procédure **calcule ou modifie des valeurs** qui seront **retournées à l'extérieur**.
- Ils ne sont pas initialisés à l'entrée, mais leur valeur est définie à l'intérieur de la procédure.
- **Exemple :**

```
Procédure calculerCarre(x : entier, carre : entier)
Début
    carre ← x * x
FinProcédure
```

- Ici, carre est un **paramètre de sortie** : la valeur calculée est transmise à l'extérieur.
- Les paramètres d'Entrée/sortie sont précédés par « S »

Types de parametres

Paramètres d'entrée/sortie (INOUT ou IN/OUT)

- La procédure reçoit une valeur, peut la modifier, et la modification est visible à l'extérieur.
- Exemple :

```
Procédure incrementer(valeur : entier)
Début
    valeur ← valeur + 1
FinProcédure
```

- Si valeur est un paramètre INOUT, son nouveau contenu est pris en compte après l'appel.
- Les paramètres d'Entrée/sortie sont précédés par « ES »

Exemple

```
Algorithme test
Variables m,n:entier
Procedure incrementer(E x:entier, E/S y: entier)
Début
    x ← x+1
    y ← y+1
    Ecrire( x,y)
Fin
/*********************************/
Début
    n ← 3
    m ← 3
    Incrementer(n,m)
    Ecrire(n,m)
Fin
```

Exercice : Ecrire un algorithme qui permet d'échanger les valeurs de a et b si a>b

Algorithme Permutation
Variables a,b:entier
Procedure **permuter**(**E/S** x:entier, **E/S** y: entier)
Variable tmp:entier
Début
tmp ← x
x ← y
y ← tmp
Fin

Début
Ecrire ("Entrez deux entiers")
Lire(a,b)
Si (a>b) alors
permuter(a,b)
Finsi
Ecrire(a,b)
Fin

Variables locales et globales

- Une **variable locale:** n'est connue qu'à l'intérieur de la fonction(procédure) où elle a été définie. Elle est créée à l'appel de la fonction (procédure) et détruite à la fin de son exécution
- Une **variable globale:** est déclarée dans la partie déclaration du programme principal, elle est connue par toutes les procédures et les fonctions.

Les fonctions récursives

Fonction (procédure) **récursive**

- On appelle fonction (procédure) **récursive** toute fonction (procédure) qui s'appelleelle même.
- Exemple : la fonction factorielle : n!=n*(n-1)!

```
Fonction fact(n:entier) :entire
Début
    Si (n=0) alors
        Retourner 1
    Sinon
        Retourner n*fact(n-1)
Fin
```

```
Algorithme CalculFactoriel
Fonction fact(n : entier) : entier
Début
    Si (n = 0) Alors
        Retourner 1
    Sinon
        Retourner n * fact(n - 1)
    FinSi
FinFonction
/**************
Variables x, resultat : entier
Début
    Ecrire("Entrez un nombre : ")
    Lire(x)
    resultat ← fact(x)
    Ecrire("Le factoriel de ", x, " est : ", resultat)
Fin
```

act(4)
→ 4 * fact(3)
→ 4 * (3 * fact(2))
→ 4 * (3 * (2 * fact(1)))
→ 4 * (3 * (2 * (1 * fact(0))))
→ 4 * 3 * 2 * 1 * 1 = 24

Exemple 2 : suite de Fibonacci

$$F_0 = 0$$
$$F_1 = 1$$
$$F_i = F_{i-1} + F_{i-2} \qquad \text{pour i > 1}$$

```
FONCTION fibo (n:entier):entier
 DEBUT
    SI (n=0) ou (n=1) ALORS
        RETOURNER (n)
    SINON
    RETOURNER (fibo(n-1)+fibo(n-2))
 FIN
```

Les tableaux

Tableaux à une seule dimension
Problème

- Nous voulons donner la main pour saisir les notes de 100 étudiants, ensuite les afficher, calculer la moyenne , le max et le min des valeurs saisies.
- Déclarer 300 variables ???
- **Solution :**

Déclarer un tableau de 300 cases de réel

Tableaux à une seule dimension
Définition

- Un tableau est un ensemble de variables simples de mêmes types.
- Ces variables sont identifiées par un même nom mais un indice différent indiquant la position de la donnée dans le tableau.

Déclaration d’un tableau

- Un tableau est déclaré par le mot « **Tableau** », en spécifiant la taille du tableau **n** (le nombre des éléments) et le type des données du tableau.
- **Syntaxe**
- T1: Tableau [1..10] de entier
- T2: Tableau [0..30] de réel
- T3: Tableau [5..10] de caractères

Accès aux éléments d'un tableau à une seule dimension

- Pour accéder à un élément du tableau on spécifie le nom du tableau et l'indice de l'élément.
- **Exemple :**

Algorithme tableau

Variables T: Tableau [1..5] de entier

Début

T [1] ← 12

T [2] ← 5

T [3] ← 9

T [4] ← 4

T [5] ← T [1] + T [2] + T [3] +T [4]

Ecrire (T [5])

Fin

Saisie des éléments d'un tableau

Algorithme saisi
Variables T: Tableau [1..100] de entier
i : entier
Début
Repeter
    Ecrire("Entrez le nombre d'éléments ")
    Lire (n)
Jusqu'à (n <=100)
Redim T[n] // *crée un* ***tableau dynamique*** *dont la taille dépend de la valeur saisie de n*
Pour i allant de 1 à n **faire**
    Ecrire ("Entrez l'élément" , i)
    Lire (T [i])
FinPour
Fin

Affichage d’un tableau rempli

Algorithme Affichage

Variables T: Tableau [1..100] de entier

i : entier

Début

Pour i allant de 1 à 100 **faire**

Ecrire ("L’élément " , i , "est" , T [i])

FinPour

Fin

Tableaux à 2 dimensions

- **Définition:**

Un tableau à 2 dimensions est interprété comme une combinaison de plusieurs tableaux à une seule dimension

Déclaration d’un tableau à deux dimensions

- **Syntaxe** Nom-Tableau : **Tableau** [1..n] [1..m] **de** Types des éléments
- **Exemples :** T : **Tableau** [1..3] [1..4] **de** entier

Accès aux éléments d'un tableau à 2 dimensions

- Pour accéder à un élément du tableau à 2 dimensions on spécifie le nom du tableau, le numéro de la ligne et le numéro de la colonne .
- **Syntaxe : T [i] [j]**

Avec i : numéro de la ligne, j : numéro de la colonne

Saisie des éléments d'un tableau à deux dimensions

Algorithme saisi_affichage

Variables T: Tableau [1..10] [1..10] de entier

Variables i,j : entier

Début

Repeter

Ecrire("Entrez le nombre de ligne et le nombre de colonnes ")

Lire (n,m)

Jusqu'à (n <=10 et m <=10)

Redim T[n] [m]

Pour i allant de 1 à n **faire**

Pour j allant de 1 à m **faire**

Ecrire ("Entrez l'élément" , i,j)

Lire (T [i] [j])

FinPour

FinPour

Fin

Affichage des éléments d'un tableau à deux dimensions

```
Algorithme saisi_affichage
Variables T: Tableau [1..10] [1..10] de entier
Variables i,j : entier
Début
Pour i allant de 1 à n faire
    Pour j allant de 1 à m faire
        Ecrire("T[", i, "][", j, "] = ", T[i][j])
    FinPour
FinPour
Fin
```

Exercices

- Ecrire un algorithme qui calcule la moyenne de 340 notes saisies .
- Ecrire un algorithme qui vérifie si un tableau est trié.
- Ecrire un algorithme qui calcule la somme de 2 matrices